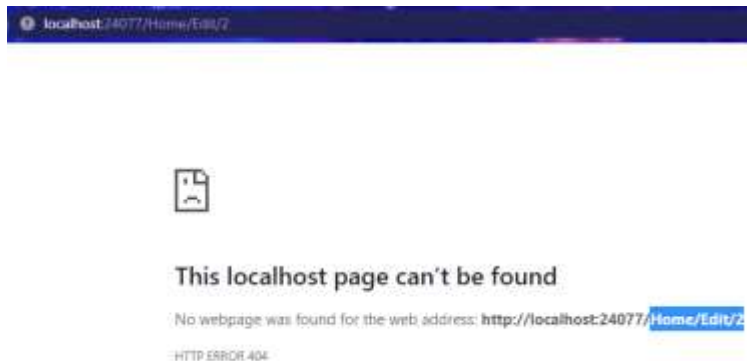


56 – HTTP post edit action

Към момента имаме списък със служители, възможност да го редактираме, но при натискане на Update btn, връща 404 резултат, т.к. няма страница, която да изведе променените данни, т.е. разполагаме само с Get – Edit, но не и с Post – Edit. Не сме го направили още.



```

Edit.cshtml | HomeController.cs | Index.cshtml
EmployeeManagement.Controllers.HomeController Edit(int id)
[HttpGet]
0 references
public ActionResult Edit(int id)
{
    Employee employee = _employeeRepository.GetEmployee(id);
    EmployeeEditViewModel employeeEditViewModel = new EmployeeEditViewModel
    {
        Id = employee.Id,
        Name = employee.Name,
        Email = employee.Email,
        Department = employee.Department,
        ExistingPhotoPath = employee.PhotoPath
    };
    return View(employeeEditViewModel);
}

```

HttpPost ще бъде сходен с Create, така че го копираме и изменяме.

```

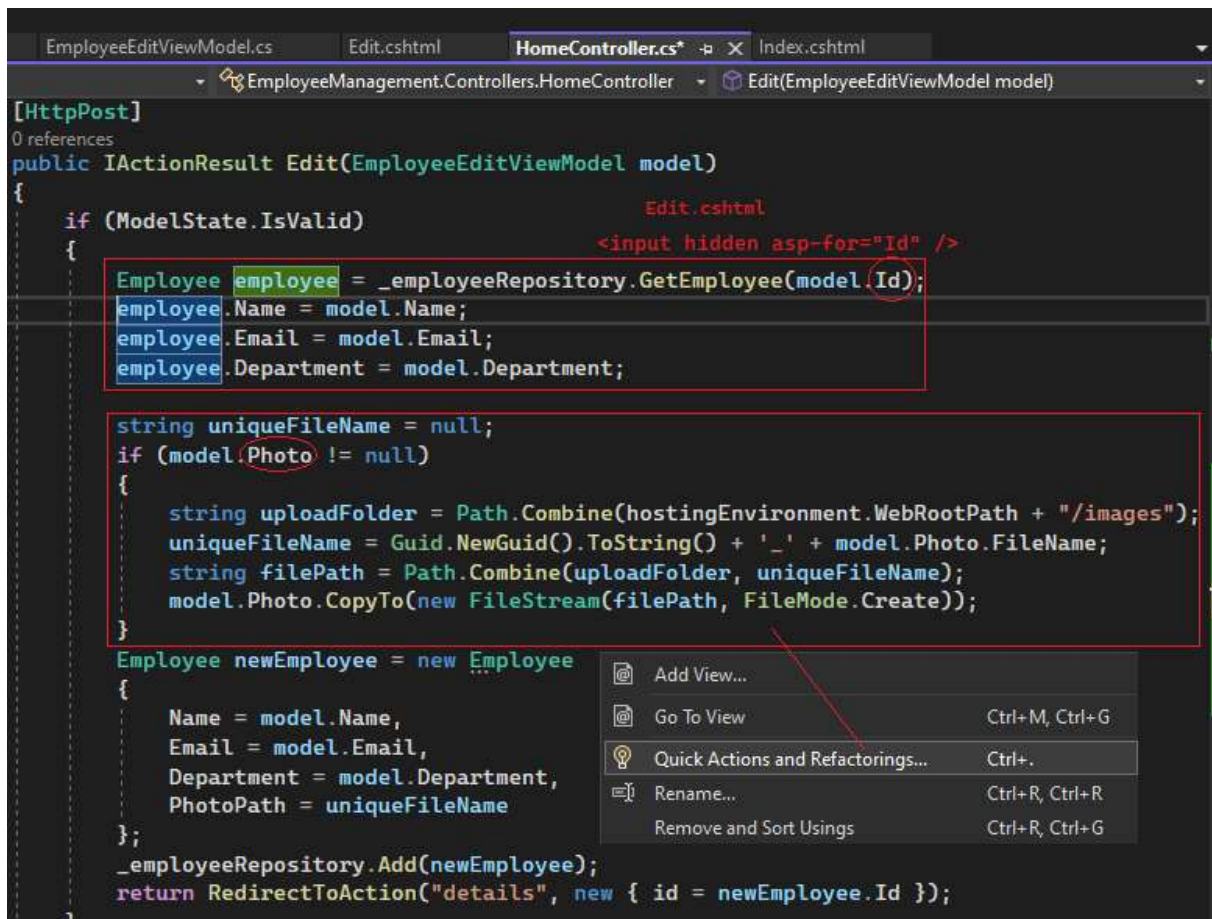
[HttpPost]
0 references
public IActionResult Edit(EmployeeEditViewModel model)
{

```

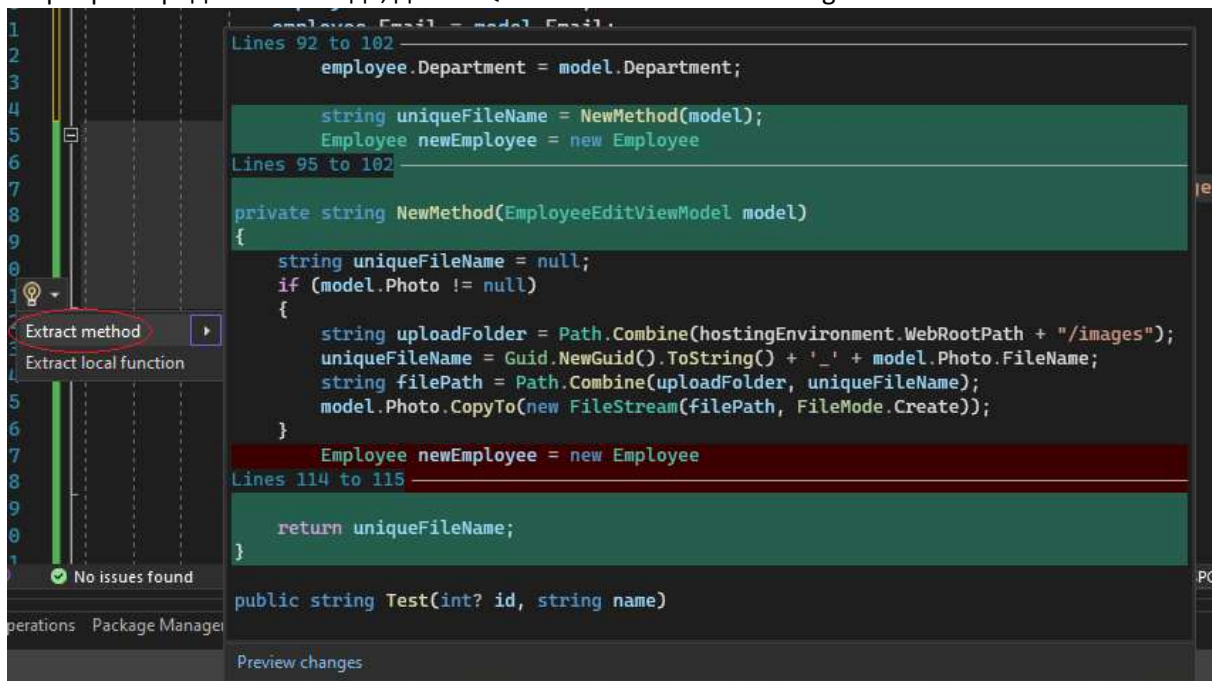
В предходното видео създадохме

EmployeeEditViewModel = EmployeeCreateViewModel + PhotoPath.

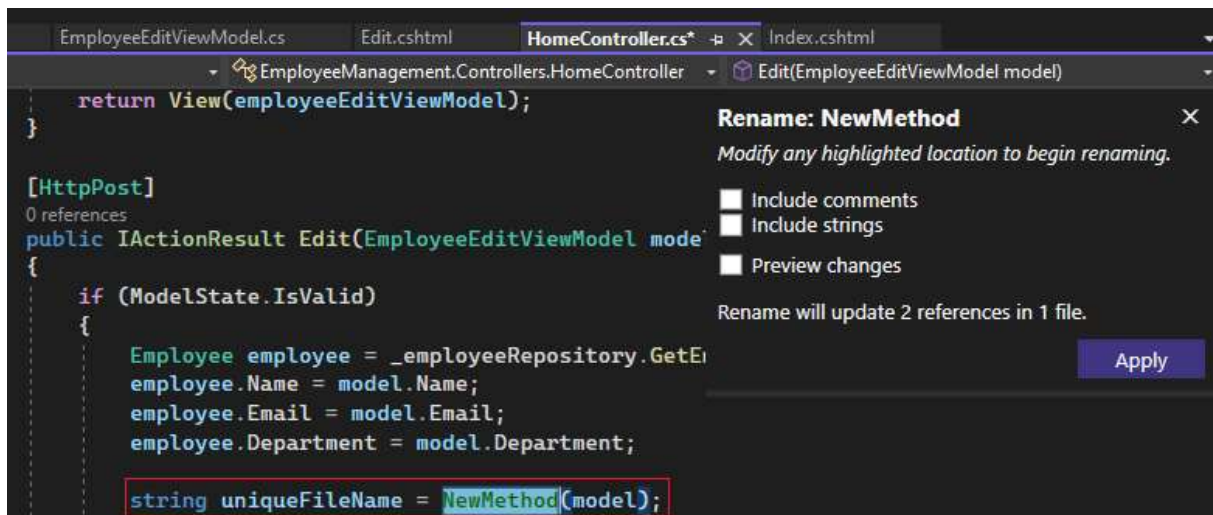
Ако валидацията е изпълнена, искаме да извлечем наличните данни от БД. В противен случай да се върне страницата с return View(); Извличането става с присвояване на Id от model -> employee, което е достъпно заради Input hidden от Edit.cshtml. Понеже кодът за снимката е еднакъв за Create и Edit, избираме да го откъснем в отделен блок код, който да извикаме на двете места, вместо да го повтаряме същият чрез д.б. Quick Actions and Refactoring.



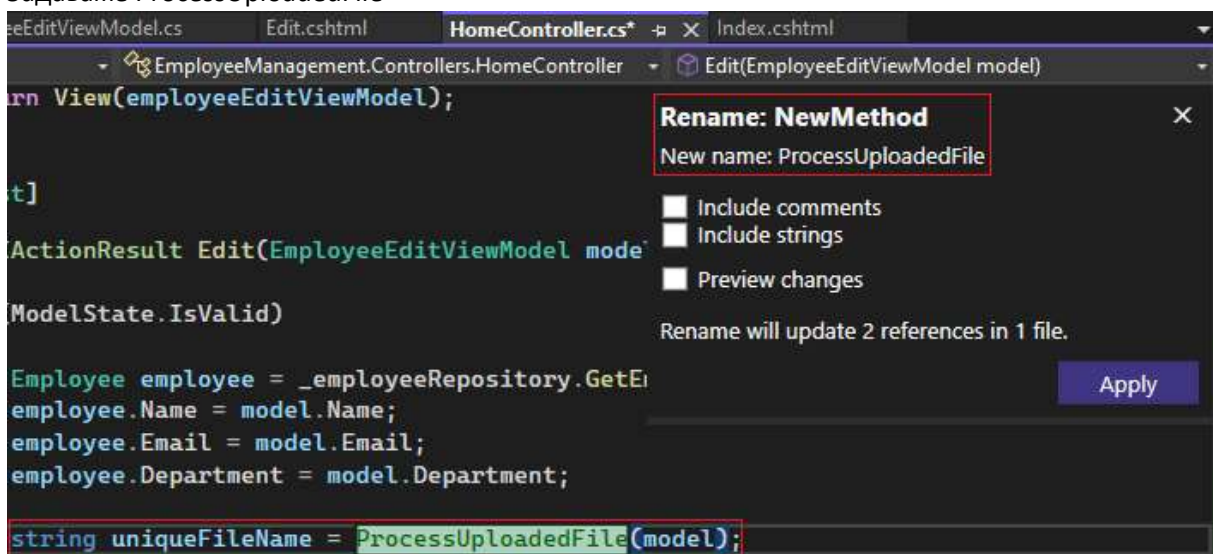
Маркираме редовете на кода, д.б. -> Quick Actions and Refactoring -> Extract method:



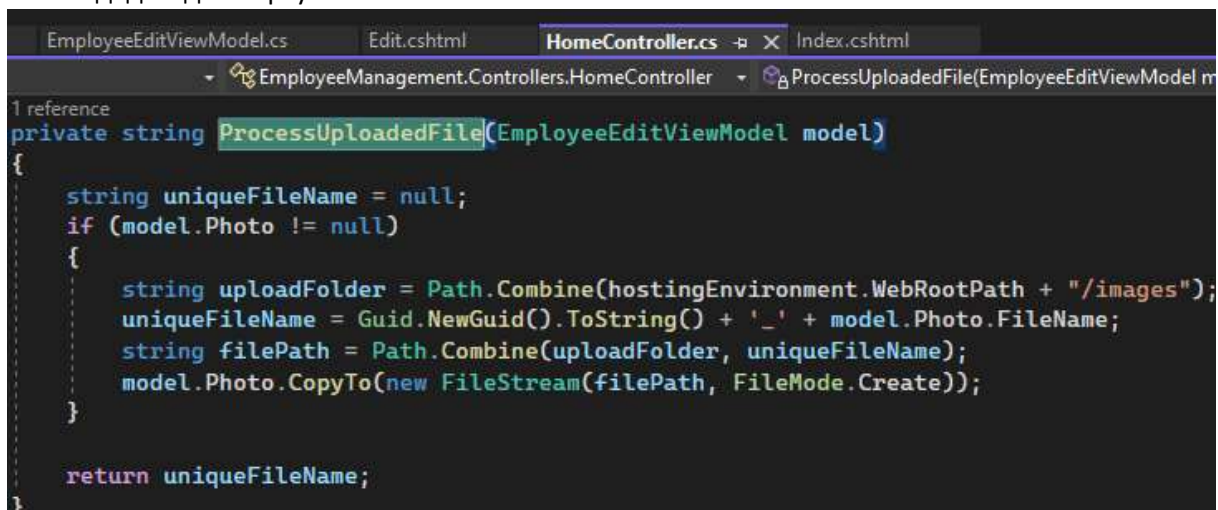
Целият код се замести с един ред, в който трябва да зададем ново име на метода, тип string



Задаваме ProcessUploadedFile



Когато дадем д.б. върху него -> Go to Definition:



Виждаме, че методът е изведен със същите редове код като последен в същия HomeController. За да спестим дубликата на всичките редове код с новия метод, който вече имаме, отиваме да ги заменим в метод Create:


```

[HttpPost]
0 references
public IActionResult Create(EmployeeCreateViewModel model)
{
    if (ModelState.IsValid)
    {
        string uniqueFileName = null;
        if (model.Photo != null)
        {
            string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images");
            uniqueFileName = Guid.NewGuid().ToString() + '_' + model.Photo.FileName;
            string filePath = Path.Combine(uploadFolder, uniqueFileName);
            model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
        }
    }
}

```

Виждаме, че на променлива тип string uniqueFileName вместо null, е присвоен новият метод с подадена стойност model:

```

string uniqueFileName = ProcessUploadedFile(model);
Employee newEmployee = new Employee
{
    Name = model.Name,
    Email = model.Email,
}

```

CS1503: Argument 1: cannot convert from 'EmployeeManagement.ViewModels.EmployeeCreateViewModel' to 'EmployeeManagement.ViewModels.EmployeeEditViewModel'

model е с червено подчертаване, защото не може да конвертира от EmployeeCreateViewModel->EmployeeEditViewModel. Понеже единия клас наследява другия (Edit наследява от Create), всичко, което е необходимо да направим, е върху метода ProcessUploadedFile д.б. -> Go to Definition -> където да сменим единия с другия модел:

```

2 references
private string ProcessUploadedFile(EmployeeEditViewModel model)

```

Т.е. от по-висшестоящия сложен да преминем към по-нисшия.

Ако разгледаме в този вид кода, заради наследяванията, на метод Create подаваме EmployeeCreateViewModel, докато на Edit подаваме EmployeeEditViewModel.

```

[HttpPost]
0 references
public IActionResult Create(EmployeeCreateViewModel model)
{
}

```

и

```

[HttpPost]
0 references
public IActionResult Edit(EmployeeEditViewModel model)
{
}

```

Ако не се предостави нова снимка, искаме да запазим съществуващата.

```

EmployeeEditViewModel.cs Edit.cshtml HomeController.cs Index.cshtml
<div class="custom-file">
    <input multiple asp-for="Photo" class="form-control custom-file-input" />
    <label class="custom-file-label">Click here to change photo</label>
</div>

```

Това може да стане, като погледнем в Edit.cshtml -> Photo property is not NULL.

Тогава искаме да се копира картинката в папката на компютъра чрез ProcessUploadedFile.

```

        if (model.Photo != null)
        {
            employee.PhotoPath = ProcessUploadedFile(model);
        }
        _employeeRepository.Update(employee);
        return RedirectToAction("index");
    }
    return View();
}

```

Остана да проверим дали вече е прикачена снимка и ако има такава, искаме да я изтрием.

Edit.cshtml × HomeController.cs EmployeeEdit

>

```

[HttpPost]
0 references
public IActionResult Edit(EmployeeEditViewModel model)
{
    if (ModelState.IsValid)
    {
        Employee employee = _employeeRepository.GetEmployee(model.Id);
        employee.Name = model.Name;
        employee.Email = model.Email;
        employee.Department = model.Department;
        if (model.Photo != null)
        {
            if(model.ExistingPhotoPath != null)
            {
                string filePath = Path.Combine(hostingEnvironment.WebRootPath,
                                                "images", model.ExistingPhotoPath);
                System.IO.File.Delete(filePath);
            }
            employee.PhotoPath = ProcessUploadedFile(model);
        }
        _employeeRepository.Update(employee);
        return RedirectToAction("index");
    }
    return View();
}

```

```

[HttpPost]
public IActionResult Edit(EmployeeEditViewModel model)
{
    if (ModelState.IsValid)
    {
        Employee employee = _employeeRepository.GetEmployee(model.Id);
        employee.Name = model.Name;
        employee.Email = model.Email;
        employee.Department = model.Department;
        if (model.Photo != null)
        {
            if(model.ExistingPhotoPath != null)
            {
                string filePath =
Path.Combine(hostingEnvironment.WebRootPath, "images", model.ExistingPhotoPath);

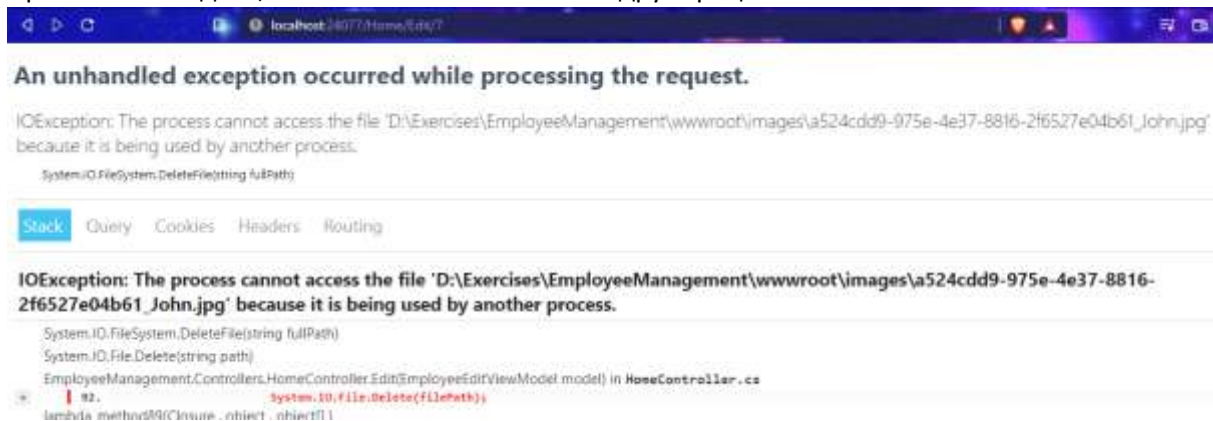
```

```

        System.IO.File.Delete(filePath);
    }
    employee.PhotoPath = ProcessUploadedFile(model);
}
_employeeRepository.Update(employee);
return RedirectToAction("index");
}
return View();
}

```

При мен обаче дава, че снимката се използва от друг процес.



Това е следващият ProcessUploadedFile. Трябва да освободим filePath, която го използва.

```

private string ProcessUploadedFile(EmployeeCreateViewModel model)
{
    string uniqueFileName = null;
    if (model.Photo != null)
    {
        string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath + "/images");
        uniqueFileName = Guid.NewGuid().ToString() + "_" + model.Photo.FileName;
        string filePath = Path.Combine(uploadFolder, uniqueFileName);
        model.Photo.CopyTo(new FileStream(filePath, FileMode.Create));
    }

    return uniqueFileName;
}

```

За да го освободим, го отделяме в цикъл, при края на който, използването се прекратява. Въвеждаме нова променлива fileStream:

```

using (var fileStream = new FileStream(filePath, FileMode.Create))
{
    model.Photo.CopyTo(fileStream);
}

```

Сега вече грешката липсва и освен Update, работи и Delete на снимката от локалния диск.

```

private string ProcessUploadedFile(EmployeeCreateViewModel model)
{
    string uniqueFileName = null;
    if (model.Photo != null)
    {
        string uploadFolder = Path.Combine(hostingEnvironment.WebRootPath
+ "/images");
        uniqueFileName = Guid.NewGuid().ToString() + "_" +
model.Photo.FileName;
        string filePath = Path.Combine(uploadFolder, uniqueFileName);

```

```

        using (var fileStream = new FileStream(filePath,
        FileMode.Create))
        {
            model.Photo.CopyTo(fileStream);
        }

        return uniqueFileName;
    }

```

При това можем да избираме вече съществуваща снимка.

HttpPost Edit action

```

[HttpPost]
public IActionResult Edit(EmployeeEditViewModel model)
{
    if (ModelState.IsValid)
    {
        Employee employee = _employeeRepository.GetEmployee(model.Id);
        employee.Name = model.Name; employee.Email = model.Email;
        employee.Department = model.Department;
        if (model.Photo != null)
        {
            if (model.ExistingPhotoPath != null)
            {
                string filePath = Path.Combine(hostingEnvironment.WebRootPath,
                "images", model.ExistingPhotoPath);
                System.IO.File.Delete(filePath);
            }
            employee.PhotoPath = ProcessUploadedFile(model);
        }
        Employee updatedEmployee = _employeeRepository.Update(employee);
        return RedirectToAction("index");
    }
    return View(model);
}

```

ProcessUploadedFile Private Method

```

private string ProcessUploadedFile(EmployeeCreateViewModel model)
{
    string uniqueFileName = null;

    if (model.Photo != null)
    {
        string uploadsFolder = Path.Combine(hostingEnvironment.WebRootPath, "images");
        uniqueFileName = Guid.NewGuid().ToString() + "_" + model.Photo.FileName;
        string filePath = Path.Combine(uploadsFolder, uniqueFileName);
        using (var fileStream = new FileStream(filePath, FileMode.Create))
        {
            model.Photo.CopyTo(fileStream);
        }

        return uniqueFileName;
    }
}

```